基于共享内存的BMP图像并行卷积



2025

问题回顾

- 任务：对BMP图像进行高斯卷积处理，实现图像模糊处理
- 输入图像：timg.bmp（RGB三通道，8位深度）
- 卷积核：5×5的高斯核，标准差1.5
- 边缘处理：零填充
- 分别采用Pthread和OpenMP两种方式进行加速运算
- 对比 2 核与 4 核的并行加速比与效率

2025

问题解析

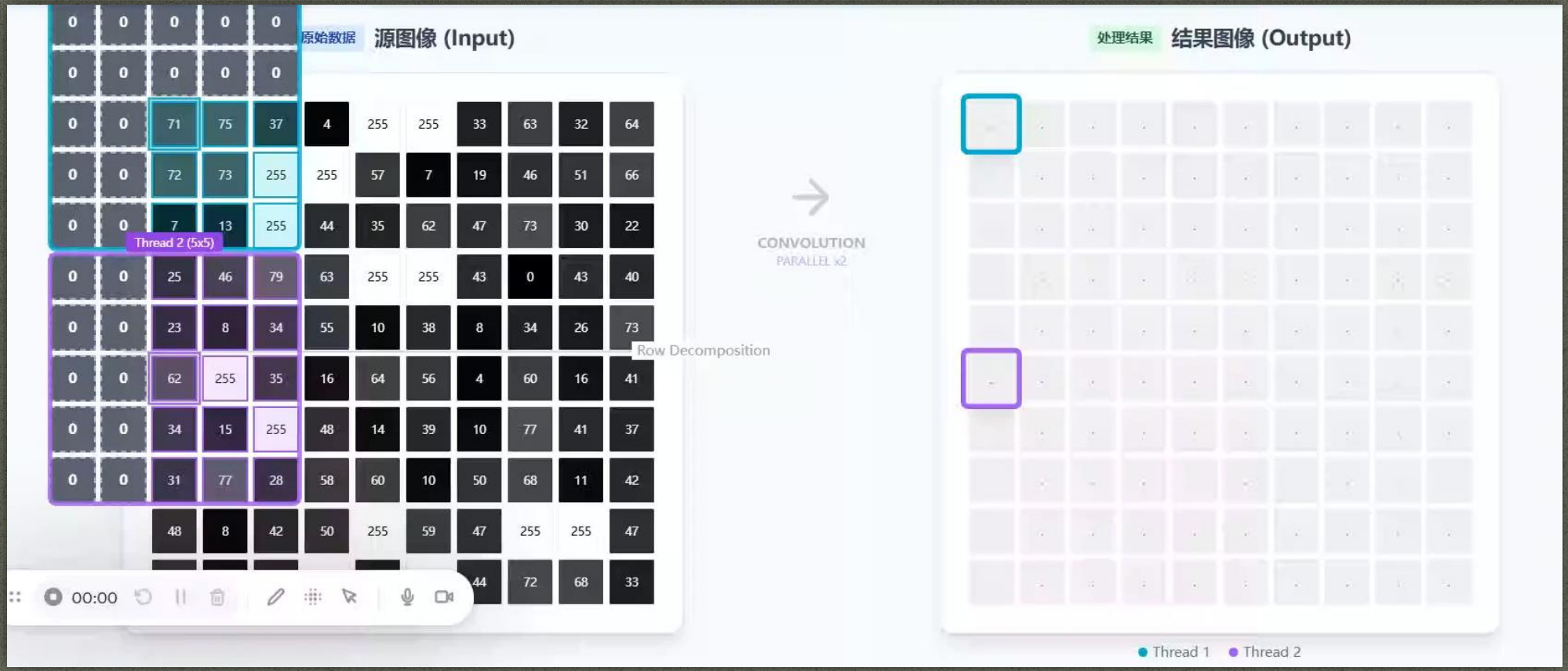
PThread > 测试程序 > gk_r2_dev1.5.txt

```
1   0.01441881 0.02808402 0.03507270 0.02808402 0.01441881
2   0.02808402 0.05470020 0.06831229 0.05470020 0.02808402
3   0.03507270 0.06831229 0.08531173 0.06831229 0.03507270
4   0.02808402 0.05470020 0.06831229 0.05470020 0.02808402
5   0.01441881 0.02808402 0.03507270 0.02808402 0.01441881
```

高斯卷积特点：卷积核遵循高斯分布，中间高，四周低，对称（可分离/降维）
通常用于对图像的平滑、模糊、降噪处理



如何并行



这里以2核为例，把原图像划分为两个部分，每个部分分别用相同的卷积核完成卷积

Pthread方案

- 采用基于共享内存架构的数据并行模式，核心思想：
  - 显示控制：代码编写负责线程的生命周期
  - 无锁设计：利用数据独立的特性，避免互斥锁的开销
  - 主从模式
    - 主线程：负责IO，内存分配，任务划分
    - 工作线程：负责计算各自部分

- 数据划分：按行区域划分为对应核数的图像部分（静态划分，公式计算）
- 特点：任务分配是固定的，若线程 0 处理得慢，线程 1 完成后无法帮忙，存在潜在的短板效应

OpenMP方案

- 架构核心思想Fork-Join模型
  - 隐藏线程创建与销毁的细节，通过编译器指令驱动
  - 指令级并行：将串行代码通过 parallel for 直接转化为并行流
  - 隐式管理：OpenMP 运行库负责底层的任务分发

- 数据划分与调度
  - 指令应用：#pragma omp parallel for collapse(2) dynamic，将行和列双重循环合并为一个的迭代空间，极大增加了并行粒度
  - 动态调度：解决了由于 CPU 状态波动或边缘处理差异导致的微小负载不均（短板效应）

关键代码分析Pthread

```
struct ThreadData {
    int thread_id;
    int num_threads;
    BMPImage* srcImg;
    BMPImage* destImg;
    double cost_time; // 存储该线程的运行时间
};
```

原图像指针
目标图像指针

线程ID与总数

各个线程计时

计算块大小与
初始行

结束行处理，处理余数，确保覆盖

```
void* convolution_routine(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    double start_time = get_wall_time();

    int height = data->srcImg->height;
    int width = data->srcImg->width;

    // 按行划分任务
    int chunk_size = height / data->num_threads;
    int start_row = data->thread_id * chunk_size;
    int end_row = (data->thread_id == data->num_threads - 1) ? height : start_row + chunk_size;
```

关键代码分析Pthread

K_RADIUS
卷积核半径：-2 ~ 2

```
// 卷积操作
for (int y = start_row; y < end_row; ++y) {
    for (int x = 0; x < width; ++x) {
        float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;

        // 应用 5x5 核
        for (int ky = -K_RADIUS; ky <= K_RADIUS; ++ky) {
            for (int kx = -K_RADIUS; kx <= K_RADIUS; ++kx) {
                int ny = y + ky;
                int nx = x + kx;

                // 边缘节点填0
                if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
                    Pixel& p = data->srcImg->data[ny * width + nx];
                    float w = KERNEL[ky + K_RADIUS][kx + K_RADIUS];
                    sum_r += p.r * w;
```

外层循环遍历图像像素

内层循环遍历卷积核

边界判断：范围内则直接指针访问内存
如果越界直接跳过累加，等同于加上0

数据存成了一整块连续的内存块，一维方式读取

关键代码分析OpenMP

开始计时

```
double start_time = omp_get_wtime(); // OpenMP 开始计时

// #pragma omp parallel for: 自动将 for 循环的任务分配给多个线程
#pragma omp parallel for collapse(2) schedule(dynamic)
for (int y = 0; y < height; ++y) {
    for (int x = 0; x < width; ++x) {
```

核心指令：

parallel for： 告诉编译器将后面的 for 循环拆分给线程组执行

collapse(2)： 将两层嵌套循环展平成一个巨大的单层迭代空间

schedule(dynamic)： 采用动态调度策略，让执行快的线程自动领取新任务块，有效应对边缘处理带来的计算耗时波动

结果性能分析

Pthread

核心数 (p)	运行时间 (s)	加速比 (S_p)	效率 (E_p)
1	1.03232	1.000	100.0%
2	0.528216	1.95	97.5%
4	0.302241	3.41	85.25%

OpenMP

核心数 (p)	运行时间 (s)	加速比 (S_p)	效率 (E_p)
1	1.05997	1.000	100.0%
2	0.676487	1.56	76.5%
4	0.359673	2.94	73.5%

结果性能分析

- 基础版实现中，Pthread并行效率优于OpenMP
- Pthread
  - 静态划分行，无额外时间开销
  - 无锁设计，数据写入互不重叠
- OpenMP
  - 动态调度：线程每次领任务都需要访问共享队列，产生互斥锁开销
  - 隐式同步：parallel for 结束时有一个隐式的 Barrier，所有线程必须等待最慢的那个线程做完，才能一起退出，放大了短板效应

瓶颈分析

1. **计算冗余**：5×5的高斯核，对于每个像素点都要进行25次乘加运算，总浮点运算量极大

2. **访存不连续**：二维卷积需要读取邻域数据，内存访问是跳跃的，导致cache命中率低下

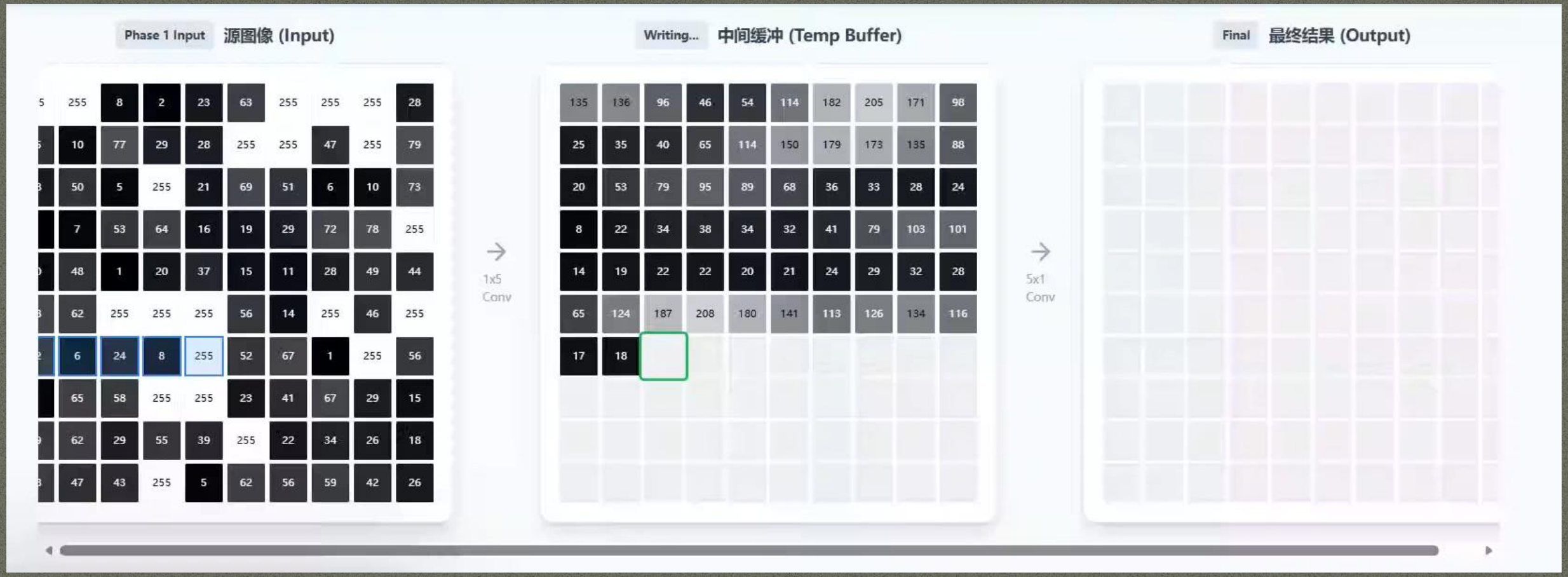
优化策略

算法优化：可分离卷积

原理：高斯核是一个线性可分矩阵，它本质上是由一个水平的一维高斯向量和一个垂直的一维高斯向量外积生成的，两个向量相等

$$C = A \otimes B = \begin{pmatrix} a_1b_1 & a_1b_2 & \cdots & a_1b_n \\ a_2b_1 & a_2b_2 & \cdots & a_2b_n \\ \vdots & \vdots & \ddots & \vdots \\ a_mb_1 & a_mb_2 & \cdots & a_mb_n \end{pmatrix} = A = \begin{pmatrix} a_1 & a_2 & \cdots & a_m \end{pmatrix} \quad B = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

优化策略


Phase 1 Input
源图像 (Input)
Writing...
中间缓冲 (Temp Buffer)
Final
最终结果 (Output)
1x5
Conv
5x1
Conv

优化收益分析

理论优化效果

维度	二维直接卷积 (优化前)	可分离卷积 (优化后)	收益
计算复杂度	O(K × K)	O(K + K)	
单点运算量	25 次乘加	5 + 5 = 10 次乘加	**减少 60%**
访存模式	二维邻域（跳跃读取）	一维线性（连续读取）	**缓存更友好**
理论加速比	1.0x	**2.5x**	

对于访存模式，垂直卷积部分的确没有连续访存，依然是跨行

但是大幅降低了带宽压力，原本一次需要送入25个元素进入缓存，现在水平和垂直一共10个，缓解了大量访存的压力

优化代码分析

遍历图像

```
// 对每一行，应用 1x5 卷积核
for (int y = start_row; y < end_row; ++y) {
    for (int x = 0; x < width; ++x) {
        float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;

        for (int k = -K_RADIUS; k <= K_RADIUS; ++k) {
            int nx = x + k;
            float w = KERNEL_1D[k + K_RADIUS];

            // 边界 Zero Padding
            if (nx >= 0 && nx < width) {
                Pixel& p = data->srcImg->data[y * width + nx];
                sum_r += p.r * w;
```

线程同步

```
// 必须等待所有线程完成水平卷积，才能开始垂直卷积
// 因为垂直卷积需要读取上下行的中间结果
pthread_barrier_wait(data->barrier);
```

完成水平卷积

完成垂直卷积

```
// 对每一列，应用 5x1 卷积核
for (int y = start_row; y < end_row; ++y) {
    for (int x = 0; x < width; ++x) {
        float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;

        for (int k = -K_RADIUS; k <= K_RADIUS; ++k) {
            int ny = y + k;
            float w = KERNEL_1D[k + K_RADIUS];

            // 边界 Zero Padding
            if (ny >= 0 && ny < height) {
                PixelFloat& p = (*data->tempImg)[ny * width + x];
                sum_r += p.r * w;
```

结果性能分析

Pthread

核心数 (p)	运行时间 (s)	加速比 (S_p)	效率 (E_p)
1	0.397706	1.000	100.0%
2	0.202889	1.95	97.5%
4	0.126152	3.15	78.75%

OpenMP

核心数 (p)	运行时间 (s)	加速比 (S_p)	效率 (E_p)
1	0.39544	1.000	100.0%
2	0.199691	1.98	99%
4	0.103255	3.82	95.5%

结果性能分析

- 优化版实现a中，OpenMP并行效率优于Pthread
- 提升显著，四核状态下最快达到0.103255，相比之前的0.302241进一步提升
- 为什么OpenMP更快
  - #pragma omp parallel for collapse(2) schedule(static)，基础版中使用动态调度，存在额外开销，领取任务本身需要时间，在可分离卷积情况下计算更加规整，行列计算量一致，在这种情况下，静态调度是最优解，在编译/启动时就一次性把任务平均分好，运行时零调度开销
- 其余优化：转置中间矩阵，让垂直卷积也能按顺序访存，但实际效果更慢，转置本身也需要极大的开销，搬运数据的耗时远远超过了计算节省的时间

感谢聆听

2025